School of Future Tech

Case Study Report

on

Insurance claim system

by

Rudra mahesh yadav

150096725217

Index

1. Introduction to the Case Study
2. Problem Statement / Case Background (Abstract)
3. Case Study Design
4. Methods & Algorithms / Technology Applied
5. Implementation Details and Snapshots
6. Results and Conclusion
7. References

1. Introduction to the Case Study

In the healthcare and insurance industry, processing claims efficiently is critical. Manual processing is slow and error-prone. This case study presents a Health Insurance Claim Processing System built in C++ that automates the main steps of claim management.

The system allows operators to create insurance policies, submit claims, automatically validate those claims using business rules, and generate reports — all through a simple console menu. It demonstrates how structured programming and rule-based logic can solve real-world administrative problems.

2. Problem Statement / Case Background (Abstract)

Background

Insurance companies process large volumes of claims daily. Without automation, enforcing business rules consistently is difficult. A rule-based automated system can validate claims uniformly, reduce errors, and speed up processing.

Abstract

This case study implements a Health Insurance Claim Processing System in C++. The system manages policies and claims in memory, simulating a backend database. Claims are submitted against policies and then automatically validated by an engine that checks three business rules: policy activity, coverage limits, and valid claim amounts. Processed claims are displayed in a formatted report. The project uses structs, vectors, and modular functions to maintain a clean and extendable codebase.

3. Case Study Design

The system is designed as a modular, menu-driven console application with five main components:

1. Policy Management — Create policies with a unique ID, holder name, type (Gold/Silver), and coverage limit.
2. Claim Submission — Submit claims against existing policies. Claims start with a Pending status.
3. Rule-Based Validation — Automatically process all pending claims against three business rules.
4. Reporting — Display a formatted table showing every claim's ID, policy, amount, status, and reason.
5. Main Menu — A loop-driven interface allowing the user to navigate all features until exit.
    -

4. Methods & Algorithms / Technology Applied

Rule-Based Validation (validateClaim)

This function is the core of the system. It applies three rules in order:

- Rule 1 — Policy must be active. Inactive policies result in an immediate rejection.
- Rule 2 — Claim amount must not exceed the policy's maximum coverage limit.
- Rule 3 — Claim amount must be greater than zero.

If all three rules pass, the claim is marked APPROVED. Otherwise it is REJECTED with a reason.

Data Structures

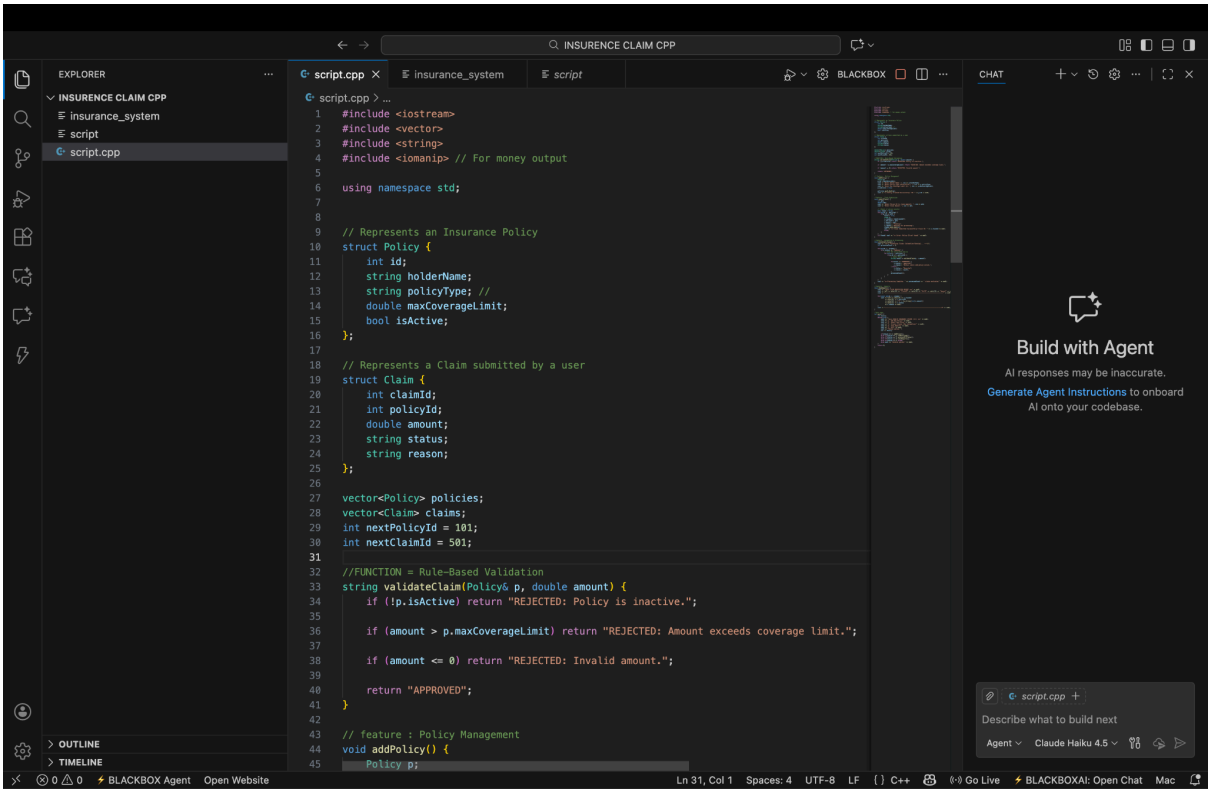
- struct Policy — Stores ID, holder name, type, coverage limit, and active status.
- struct Claim — Stores claim ID, linked policy ID, amount, status, and reason.
- vector<Policy> and vector<Claim> — Dynamic arrays used as in-memory storage.
    -

Technology Stack

- Language: C++ (C++11 or later)
- Libraries: <iostream>, <vector>, <string>, <iomanip>
- Environment: Any C++ compiler (g++, Visual Studio, Code::Blocks, etc.)
- Storage: In-memory vectors (no file or database required)
    -

5. Problem Statement / Case Study Implementation Details and Snapshots.

- **5. Implementation Details and Snapshots**
- The entire program is in a single file (script.cpp) with the following functions:
- **addPolicy()** Asks the user for a holder name, policy type, and coverage limit. Assigns an auto-incremented ID starting at 101 and saves the policy to the global vector.
- **submitClaim()** Asks for a policy ID and claim amount. Searches for the policy and if found, creates a new claim with status Pending. Displays an error if the policy ID does not exist.
- **processAllClaims()** Loops through all pending claims, finds each claim's policy, and calls validateClaim() to apply the rules. Updates each claim's status and reason, then prints how many were evaluated.
- **showReports()** Prints a formatted table of all claims using setw() for alignment. Columns: Claim ID, Policy ID, Amount, Status, Reason.
- **main()** Runs a menu loop with five options: Add Policy, Submit Claim, Process Claims, View Rep



INSURENCE CLAIM CPP

EXPLORER

INSURENCE CLAIM CPP
- insurance_system
- script
- script.cpp

script.cpp > ...

```cpp
void addPolicy() {
    p.id = nextPolicyId++;
    cout << "Enter Holder Name: "; cin >> p.holderName;
    cout << "Enter Policy Type (Gold/Silver): "; cin >> p.policyType;
    cout << "Enter Max Coverage Limit ($): "; cin >> p.maxCoverageLimit;
    p.isActive = true;

    policies.push_back(p);
    cout << ">> Policy Created Successfully! ID: " << p.id << endl;
}

//feature : Claim Submission
void submitClaim() {
    int pId;
    double amt;
    cout << "Enter Policy ID to claim against: "; cin >> pId;
    cout << "Enter Claim Amount: "; cin >> amt;

    // (Check if policy exists)
    bool found = false;
    for(auto& p : policies) {
        if(p.id == pId) {
            found = true;
            Claim c;
            c.claimId = nextClaimId++;
            c.policyId = pId;
            c.amount = amt;
            c.status = "Pending";
            c.reason = "Waiting for processing";
            claims.push_back(c);
            cout << ">> Claim Submitted Successfully! Claim ID: " << c.claimId << endl;
            break;
        }
    }
    if(!found) cout << ">> Error: Policy ID not found." << endl;
}

//feature : Automation & Processing
void processAllClaims() {
    cout << "\n--- Processing Claims (Automation Running)... ---\n";
    int processedCount = 0;

    for(auto& c : claims) {
        if(c.status == "Pending") {
            // Find the associated policy
```

Build with Agent

AI responses may be inaccurate.

Generate Agent Instructions to onboard AI onto your codebase.

Describe what to build next

Agent · Claude Haiku 4.5

0 0 · BLACKBOX Agent · Open Website · Ln 31, Col 1 · Spaces: 4 · UTF-8 · LF · C++ · Go Live · BLACKBOXAI: Open Chat · Mac

INSURENCE CLAIM CPP

EXPLORER

INSURENCE CLAIM CPP
- insurance_system
- script
- script.cpp

script.cpp > ...

```cpp
void processAllClaims() {
    for(auto& c : claims) {
        if(c.status == "Pending") {
            for(auto& p : policies) {
                if(p.id == c.policyId) {
                    // Apply Rules
                    string result = validateClaim(p, c.amount);

                    if(result == "APPROVED") {
                        c.status = "Approved";
                        c.reason = "Within limits and policy active.";
                    } else {
                        c.status = "Rejected";
                        c.reason = result;
                    }
                    processedCount++;
                }
            }
        }
    }
    cout << ">> Processing Complete. " << processedCount << " claims evaluated." << endl;


//feature : Reports
void showReports() {
    cout << "\n=== CLAIM PROCESSING REPORT ===" << endl;
    cout << left << setw(10) << "ClaimID" << setw(10) << "PolID" << setw(10) << "Amount" <
    cout << "----------------------------------------------------------------------" << en

    for(const auto& c : claims) {
        cout << left << setw(10) << c.claimId
             << setw(10) << c.policyId
             << setw(10) << ("$" + to_string((int)c.amount))
             << setw(13) << c.status
             << c.reason << endl;
    }
    cout << "----------------------------------------------------------------------" << enc


//main menu
int main() {
    int choice;
    while(true) {
        cout << "\n=== HEALTH INSURANCE SYSTEM (C++) ===" << endl;
        cout << "1. Add New Policy" << endl;
```

Build with Agent

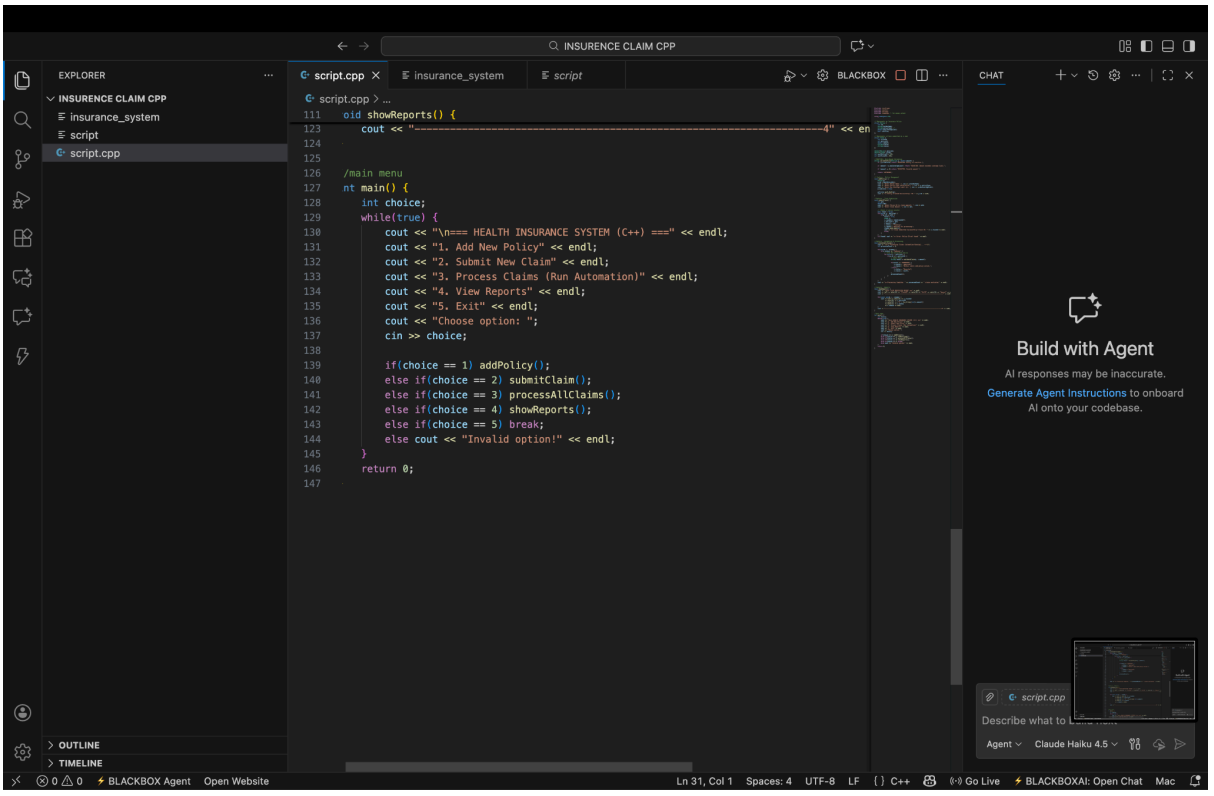
AI responses may be inaccurate.

Generate Agent Instructions to onboard AI onto your codebase.

Describe what to build next

Agent · Claude Haiku 4.5

0 0 · BLACKBOX Agent · Open Website · Ln 31, Col 1 · Spaces: 4 · UTF-8 · LF · C++ · Go Live · BLACKBOXAI: Open Chat · Mac



orts, and Exit.

-

Sample Interaction Flow

- Create Gold policy for Alice, limit $5,000 — Policy ID: 101
- Create Silver policy for Bob, limit $2,000 — Policy ID: 102
- Submit claim of $1,500 against Policy 101 — Claim ID: 501
- Submit claim of $3,000 against Policy 102 — Claim ID: 502
- Run Process Claims — Claim 501 Approved, Claim 502 Rejected (exceeds limit)
- View Reports — table shows both claims with their status and reason

6. Results and Conclusion

Findings

- The rule-based engine correctly approves and rejects claims based on the three defined rules.
- Modular functions make the code easy to read, test, and extend.
- Auto-increment IDs ensure every record is uniquely identified.
- Formatted output using iomanip makes reports clear and readable.

Limitations

- Data is not saved to disk — all records are lost when the program closes.

- No input validation for incorrect data types.
- No support for deactivating policies or editing existing records.

Conclusion

This case study demonstrates a complete Health Insurance Claim Processing System using core C++ concepts. The project successfully integrates policy management, claim submission, automated rule-based validation, and reporting into a working console application. It can be extended with file storage, a GUI, or a database for real-world use.

7. References

- GeeksforGeeks — C++ tutorials. https://www.geeksforgeeks.org
- General domain knowledge on health insurance claim processing workflows.
- Class notes by vinaya maam